

Nama:Hafidza Tsamara Zahra
NIM:254107020034
Kelas:1G-TI
Absen:11

LAPORAN TUGAS PERTEMUAN 14

14.2.1 Percobaan 1

class mahasiswa11.java

```
package pertemuan14;

public class mahasiswa11 {

    String nim;
    String nama;
    String kelas;
    double ipk;

    public mahasiswa11(){

    }

    public mahasiswa11(String nim, String nama, String kelas, double ipk){

        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;

    }

    public void tampilInformasi(){
        System.out.println("NIM: "+this.nim+" "+
            "Nama: "+this.nama+" "+
            "Kelas: "+this.kelas+" "+
            "Ipk: "+this.ipk);
    }

}
```

class node11.java

```
package pertemuan14;

public class node11 {

    mahasiswa11 mahasiswa;
    node11 left, right;

    public node11(){
```

```

    }

    public node11(mahasiswa11 mahasiswa){
        this.mahasiswa = mahasiswa;
        left = right = null;
    }
}

```

class binaryTree11.java

```

package pertemuan14;

public class binaryTree11 {
    node11 root;

    public binaryTree11(){
        root = null;
    }

    public boolean isEmpty(){
        return root == null;
    }

    public void add(mahasiswa11 mahasiswa){
        node11 newNode = new node11(mahasiswa);
        if (isEmpty()){
            root = newNode;
        }else{
            node11 current = root;
            node11 parent = null;
            while(true){
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk){
                    current = current.left;
                    if(current == null){
                        parent.left = newNode;
                        return;
                    }
                }else{
                    current = current.right;
                    if(current == null){
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }
}

```

```

    }
}

boolean find(double ipk){
    boolean result = false;
    node11 current = root;
    while (current != null){
        if(current.mahasiswa.ipk == ipk){
            result = true;
            break;
        }else if(ipk > current.mahasiswa.ipk){
            current = current.right;
        }else{
            current = current.left;
        }
    }
    return result;
}

void traversePreOrder(node11 node){
    if(node != null){
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(node11 node){
    if(node != null){
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(node11 node){
    if(node != null){
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

```

```

}

node11 getSuccessor(node11 del){
    node11 successor = del.right;
    node11 successorParent = del;
    while(successor.left != null){
        successorParent = successor;
        successor = successor.left;
    }
    if(successor != del.right){
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

void delete(double ipk){
    if(isEmpty()){
        System.out.println("Binary tree kosong");
        return;
    }
    node11 parent = root;
    node11 current = root;
    boolean isLeftChild = false;
    while (current != null){
        if(current.mahasiswa.ipk == ipk){
            break;
        }else if(ipk < current.mahasiswa.ipk){
            parent = current;
            current = current.left;
            isLeftChild = true;
        }else if(ipk > current.mahasiswa.ipk){
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    if(current == null){
        System.out.println("data tidak ditemukan");
        return;
    }else{
        if(current.left == null && current.right == null){
            if(current == root){

```

```

        root = null;
    }else{
        if(isLeftChild){
            parent.left = null;
        }else{
            parent.right = null;
        }
    }
}
}else if(current.left == null){
    if(current == root){
        root = current.right;
    }else{
        if(isLeftChild){
            parent.left = current.right;
        }else{
            parent.right = current.right;
        }
    }
}
}else if(current.right == null){
    if(current == root){
        root = current.left;
    }else{
        if(isLeftChild){
            parent.left = current.left;
        }else{
            parent.right = current.left;
        }
    }
}
}else{
    node11 successor = getSuccessor(current);
    System.out.println("jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if(current == root){
        root = successor;
    }else{
        if(isLeftChild){
            parent.left = successor;
        }else{
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}

```

```

    }
}
}

```

class binaryTreeMain11.java

```

package pertemuan14;

public class binaryTreeMain11 {
    public static void main(String[] args) {
        binaryTree11 bst = new binaryTree11();

        bst.add(new mahasiswa11("244160121", "Ali", "A", 3.57));
        bst.add(new mahasiswa11("244160221", "Badar", "B", 3.85));
        bst.add(new mahasiswa11("244160185", "Candra", "C", 3.21));
        bst.add(new mahasiswa11("244160220", "Dewi", "D", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in odrder
traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54)?"Ditemukan":"Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22)?"Ditemukan":"Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new mahasiswa11("244160131", "Devi", "A", 3.72));
        bst.add(new mahasiswa11("244160205", "Ehsan", "D", 3.37));
        bst.add(new mahasiswa11("244160170", "Fizi", "B", 3.46));
        System.out.println("\nDaftar semua mahasiswa setelah perubahan 3
mahasiswa:");
        System.out.println("InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("PreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("PostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
    }
}

```

```
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (In Order Traversal):");
        bst.traverseInOrder(bst.root);
    }
}
```

hasil output

Daftar semua mahasiswa (in odrder traversal):
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

Pencarian data mahasiswa:
cari mahasiswa dengan ipk: 3.54 : Ditemukan
cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah perubahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46
NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

PreOrder Traversal:

NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

PostOrder Traversal:

NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37
NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85
NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57

Penghapusan data mahasiswa

jika 2 anak, current =

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (In Order Traversal):

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

14.2.2 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?
2. Untuk apakah di class Node, kegunaan dari atribut left dan right?
3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?
b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?
5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
    if (mahasiswa.ipk < current.mahasiswa.ipk){
        current = current.left;
        if(current == null){
            parent.left = newNode;
            return;
        }
    }else{
        current = current.right;
        if(current == null){
            parent.right = newNode;
            return;
        }
    }
}
```

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

jawaban pertanyaan

1. Karena Binary Search Tree memiliki aturan: data yang lebih kecil berada di kiri, sedangkan data yang lebih besar berada di kanan. Jadi saat mencari data, program tidak perlu mengecek semua node, cukup memilih ke kiri atau kanan berdasarkan nilai IPK.
2. Atribut left digunakan untuk menyimpan alamat node anak sebelah kiri, sedangkan right digunakan untuk menyimpan alamat node anak sebelah kanan.
3. a. root digunakan sebagai node utama atau akar dari tree. Dari root, seluruh node lain dapat diakses.
b. Nilai root adalah null, karena tree masih kosong.
4. Node baru akan langsung menjadi root, karena belum ada node lain di dalam tree.
5. Kode tersebut digunakan untuk menentukan posisi node baru berdasarkan nilai IPK.

parent = current; menyimpan node saat ini sebagai parent atau calon induk dari node baru.

Jika mahasiswa.ipk < current.mahasiswa.ipk, maka data baru lebih kecil, sehingga pencarian posisi dilanjutkan ke anak kiri dengan current = current.left.

Jika ternyata current == null, berarti posisi kiri kosong, maka node baru dimasukkan sebagai anak kiri dengan parent.left = newNode.

Jika IPK baru lebih besar atau sama, maka pencarian dilanjutkan ke kanan dengan current = current.right.

Jika posisi kanan kosong, node baru dimasukkan sebagai anak kanan dengan parent.right = newNode.

6. Saat node yang akan dihapus memiliki dua anak, node tersebut tidak bisa langsung dihapus karena masih memiliki cabang kiri dan kanan. Maka program mencari node pengganti menggunakan method getSuccessor().

getSuccessor() mencari node terkecil dari subtree kanan, yaitu dengan bergerak ke kanan satu kali lalu terus ke kiri sampai tidak ada anak kiri lagi. Node ini disebut successor.

Setelah successor ditemukan, successor menggantikan posisi node yang dihapus. Anak kiri dari node yang dihapus dipasang menjadi anak kiri successor, sehingga struktur Binary Search Tree tetap teratur.

14.3 Kegiatan Praktikum 2

class binaryTreeArray11.java

```
package pertemuan14;

public class binaryTreeArray11 {

    mahasiswa11[] dataMahasiswa;

    int idxLast;

    public binaryTreeArray11 () {

        this.dataMahasiswa = new mahasiswa11[10];

    }
```

```

void populateData(mahasiswa11 dataMhs[], int idxLast){

    this.dataMahasiswa = dataMhs;

    this.idxLast = idxLast;

}

void traverseInOrder(int idxStart){

    if(idxStart <= idxLast){

        if(dataMahasiswa[idxStart] != null){

            traverseInOrder(2*idxStart+1);

            dataMahasiswa[idxStart].tampilInformasi();

            traverseInOrder(2*idxStart+2);

        }

    }

}

}

```

class binaryTreeArrayMain11.java

```

package pertemuan14;

public class binaryTreeArrayMain11 {

    public static void main(String[] args) {

        binaryTreeArray11 bta = new binaryTreeArray11();

        mahasiswa11 mhs1 = new mahasiswa11("244160121", "Ali", "A", 3.57);
    }
}

```

```

        mahasiswa1 mhs2 = new mahasiswa1("244160185", "Candra", "C",
3.41);

        mahasiswa1 mhs3 = new mahasiswa1("244160221", "Badar", "B",
3.75);

        mahasiswa1 mhs4 = new mahasiswa1("244160220", "Dewi", "B", 3.35);

        mahasiswa1 mhs5 = new mahasiswa1("244160131", "Devi", "A", 3.48);

        mahasiswa1 mhs6 = new mahasiswa1("244160205", "Ehsan", "D",
3.61);

        mahasiswa1 mhs7 = new mahasiswa1("244160170", "Fizi", "B", 3.86);

        mahasiswa1[] dataMahasiswa =
{mhs1,mhs2,mhs3,mhs4,mhs5,mhs6,mhs7,null, null, null};

        int idxLast = 6;

        bta.populateData(dataMahasiswa,idxLast);

        System.out.println("\nInOrder traversal mahasiswa: ");

        bta.traverseInOrder(0);

    }
}

```

hasil output

```

InOrder traversal mahasiswa:
NIM: 244160220 Nama: Dewi Kelas: B Ipk: 3.35
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.41
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.48
NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.61
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.75
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.86

```

14.3.2 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?
2. Apakah kegunaan dari method populateData()?
3. Apakah kegunaan dari method traverseInOrder()?
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?
6. Mengapa indeks $2 \cdot \text{idxStart} + 1$ dan $2 \cdot \text{idxStart} + 2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

jawaban

1. dataMahasiswa digunakan untuk menyimpan data mahasiswa dalam bentuk array. idxLast digunakan untuk menyimpan indeks terakhir yang berisi data pada array.
2. Method populateData() digunakan untuk mengisi atau memasukkan data array mahasiswa ke dalam objek BinaryTreeArray, sekaligus menentukan indeks terakhir datanya.
3. Method traverseInOrder() digunakan untuk menampilkan data tree secara InOrder, yaitu dengan urutan: anak kiri, root, lalu anak kanan.
4. Jika $\text{idx} = 2$, maka:
Left child = $2 \cdot 2 + 1 = 5$
Right child = $2 \cdot 2 + 2 = 6$
5. Statement tersebut menunjukkan bahwa indeks terakhir yang berisi data adalah indeks ke-6, yaitu data mhs7. Jadi traversal hanya dilakukan sampai indeks 6.
6. Karena pada binary tree yang disimpan dalam array, posisi anak kiri dan anak kanan mengikuti rumus tersebut. Untuk node pada indeks i , anak kirinya berada di indeks $2i+1$, dan anak kanannya berada di indeks $2i+2$.

14.4 Tugas Praktikum

modifikasi class binaryTree11.java

```
public void addRekursif(mahasiswa11 mahasiswa) {  
  
    root = addRekursif(root, mahasiswa);  
  
}  
  
private node11 addRekursif(node11 current, mahasiswa11 mahasiswa) {  
  
    if (current == null) {  
  
        return new node11(mahasiswa);  
  
    }  
  
}
```

```

    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) {

        current.left = addRekursif(current.left, mahasiswa);

    } else {

        current.right = addRekursif(current.right, mahasiswa);

    }

    return current;

}

public void cariMinIPK() {

    if (isEmpty()) {

        System.out.println("Binary tree kosong");

        return;

    }

    node11 current = root;

    while (current.left != null) {

        current = current.left;

    }

    System.out.println("Mahasiswa dengan IPK paling kecil:");

    current.mahasiswa.tampilInformasi();

}

```

```
public void cariMaxIPK() {  
  
    if (isEmpty()) {  
  
        System.out.println("Binary tree kosong");  
  
        return;  
  
    }  
  
    node11 current = root;  
  
    while (current.right != null) {  
  
        current = current.right;  
  
    }  
  
    System.out.println("Mahasiswa dengan IPK paling besar:");  
  
    current.mahasiswa.tampilInformasi();  
  
}  
  
public void tampilMahasiswaIPKdiAtas(double ipkBatas) {  
  
    tampilMahasiswaIPKdiAtas(root, ipkBatas);  
  
}  
  
private void tampilMahasiswaIPKdiAtas(node11 node, double ipkBatas)  
{  
  
    if (node != null) {  
  
        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);  
  
    }  
  
}
```

```

        if (node.mahasiswa.ipk > ipkBatas) {

            node.mahasiswa.tampilInformasi();

        }

        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);

    }

}

```

modifikasi class binaryTreeArray11.java

```

void add(mahasiswa11 data) {

    if (idxLast == dataMahasiswa.length - 1) {

        System.out.println("Array sudah penuh");

        return;

    }

    idxLast++;

    dataMahasiswa[idxLast] = data;

}

void traversePreOrder(int idxStart) {

    if (idxStart <= idxLast) {

        if (dataMahasiswa[idxStart] != null) {

            dataMahasiswa[idxStart].tampilInformasi();

            traversePreOrder(2 * idxStart + 1);

            traversePreOrder(2 * idxStart + 2);

        }

    }

}

```

```

    }

    }

}

```

modifikasi class binaryTreeMain11.java

```

System.out.println();

    bst.cariMinIPK();

    System.out.println();

    bst.cariMaxIPK();

    System.out.println();

    System.out.println("Mahasiswa dengan IPK di atas 3.50:");

    bst.tampilMahasiswaIPKdiAtas(3.50);

}

```

modifikasi class binaryTreeArrayMain11.java

```

System.out.println();

System.out.println("PreOrder traversal mahasiswa:");

    bta.traversePreOrder(0);

```

hasil output binaryTreeMain11.java

Daftar semua mahasiswa (in odrder traversal):
 NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21
 NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54
 NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
 NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

Pencarian data mahasiswa:
 cari mahasiswa dengan ipk: 3.54 : Ditemukan

cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah perubahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21

NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37

NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54

NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

PreOrder Traversal:

NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54

NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37

NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

PostOrder Traversal:

NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46

NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57

Penghapusan data mahasiswa

jika 2 anak, current =

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (In Order Traversal):

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21

NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.37

NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.46

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

Mahasiswa dengan IPK paling kecil:

NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.21

Mahasiswa dengan IPK paling besar:

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

Mahasiswa dengan IPK di atas 3.50:

NIM: 244160220 Nama: Dewi Kelas: D Ipk: 3.54

NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.72

NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.85

hasil output binaryTreeArrayMain11.java

InOrder traversal mahasiswa:

NIM: 244160220 Nama: Dewi Kelas: B Ipk: 3.35
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.41
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.48
NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.61
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.75
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.86

PreOrder traversal mahasiswa:

NIM: 244160121 Nama: Ali Kelas: A Ipk: 3.57
NIM: 244160185 Nama: Candra Kelas: C Ipk: 3.41
NIM: 244160220 Nama: Dewi Kelas: B Ipk: 3.35
NIM: 244160131 Nama: Devi Kelas: A Ipk: 3.48
NIM: 244160221 Nama: Badar Kelas: B Ipk: 3.75
NIM: 244160205 Nama: Ehsan Kelas: D Ipk: 3.61
NIM: 244160170 Nama: Fizi Kelas: B Ipk: 3.86